



Architectures et Administration des BDD

Présenté par:

Amal HALFAOUI (Epse GHERNAOUT)

Amal.halfaoui@univ-tlemcen.dz

amal.halfaoui@gmail.com

Université Tlemcen

-Ing 3-

- PL/SQL -

Pourquoi un langage de programmation ?

SQL n'est pas un langage de programmation

- SQL permet l'interrogation d'une base de données de manière **déclarative**, et non procédurale
- Pas de variable, pas de boucle, pas de test

Pour écrire des applications, on utilise SQL associé à un langage de programmation

Présentation

- PL/SQL : **P**rocedural **L**anguage extensions to **SQL** (**S**tructured **Q**uery **L**anguage).
- Language de programmation procédural proposé par oracle pour SQL
- Clauses SQL intégrées dans le code procédural: combiner des requêtes SQL (select, insert,..) et des instructions procédurales (boucles, conditions)
- Existe dans d'autres SGBDR
 - *MySQL : PL/SQL like*
 - *Sybase et Microsoft SQL server : Transact-SQL*
 - *PostgreSQL : PL/pgSQL*
 - *DB2 (IBM) : SQL Procedural Language*

Présentation

Produit

Id_Produit	Référence	Marque	Prix	ID_Fournisseur
P1	HP600	HP	1000	1
P2	Epson 30	Epson	2300	1
P3	Epson 600	Epson	2300	1
P4	IBL 7380	IBM	4600	2
P5	Lexmark310	Lexmark	2000	3

UPDATE produit set prix = 2500 référence = 'HP600'

Si moyenne (prix) > 2000
Augmenter le prix de 5%
tous les produits de la marque Epson
Sinon
Augmenter de 3% les autres produits

Code PL/SQL

Select AVG (Prix) from Produit....

IF..... THEN

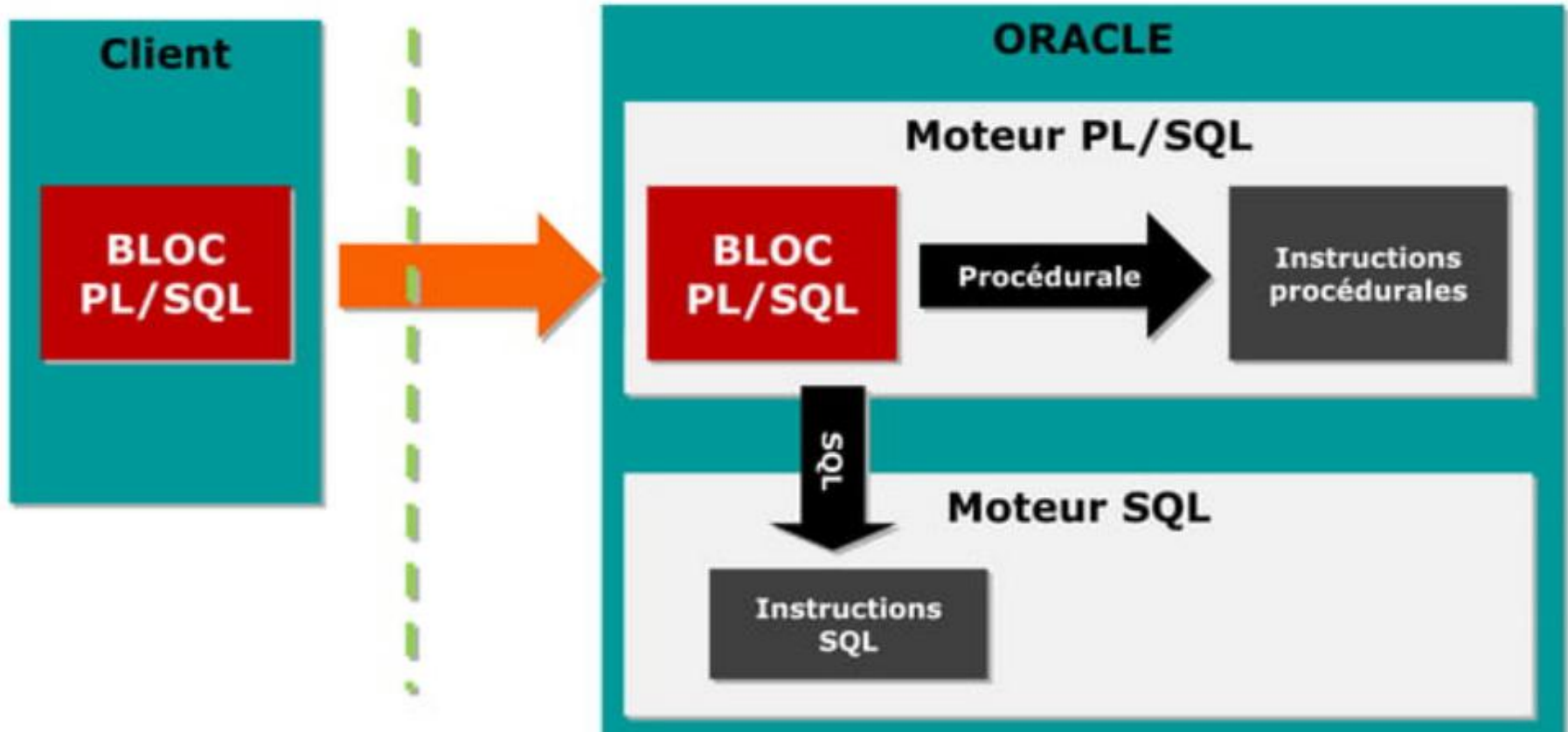
UPDATE produit set prix =....

ELSEIF

UPDATE produit set prix =....

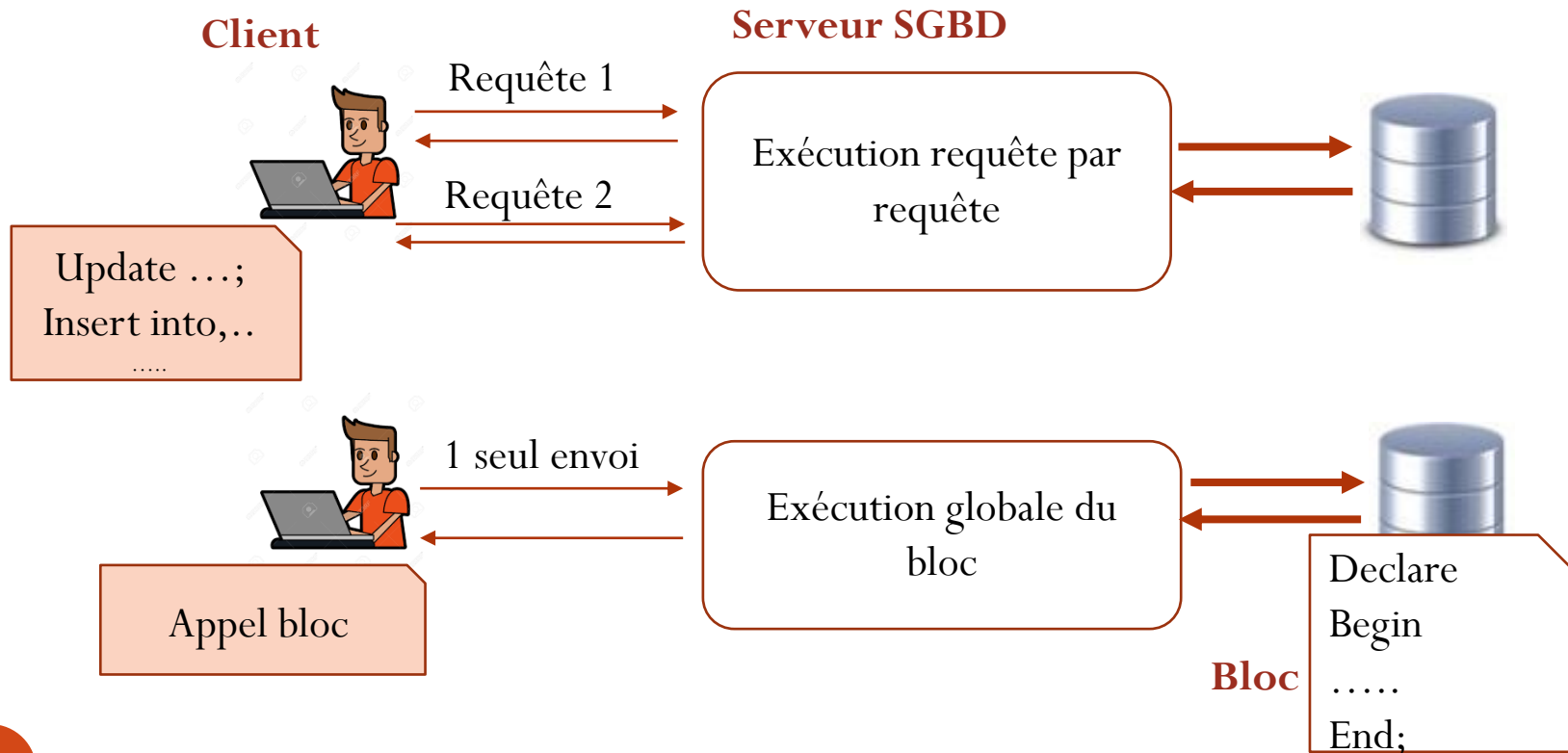
ENDIF

Architecture PL/SQL



Architecture PL/SQL

- Ecrire des programmes en **blocs** qui regroupent des requêtes SQL. **Un seul bloc** est envoyé au serveur en un seul appel
⇒ Amélioration des performances (moins de communications sur le réseau)



Avantages

- **Modularité** : Un bloc peut contenir un autre bloc, etc.
 - Un bloc peut être nommé pour devenir une procédure ou fonction (réutilisable)
 - Créer des bibliothèques de code réutilisable
- **Portabilité** : un programme PL/SQL est indépendant du système d'exploitation qui héberge le serveur Oracle.
- **Intégration avec les données des table** : Intégrer des requêtes SQL (d'interrogation (LID) et de manipulation de données (LMD)) dans un programme avec des mécanismes pour:
 - Parcourir les résultats de requêtes (**curseurs**)
 - Traiter les erreurs (**exceptions**)
 - Manipuler les données complexes (**Paquetage**), **déclencheurs**
 - Programmer des transactions (**COMMIT, ROLLBACK, SAVEPOINT**)

Fonctionnalités

- PL/SQL ne comporte pas d'instructions du LDD (ALTER, CREATE, RENAME) ni les instructions de contrôle comme GRANT et REVOKE.
- Instructions spécifiques à PL/SQL :
 - **Définition de variables**
 - **Structures de contrôle** (Traitement conditionnel et répétitif)
 - **Traitement des curseurs**
 - **Traitement des erreurs**

Structure d'un bloc

- Un programme ou une procédure PL/SQL est un ensemble de un ou plusieurs **blocs**.
- Chaque bloc comporte trois sections :
 1. **Section déclaration**
 2. **Section corps du bloc**
 3. **Section traitement des erreurs**

Structure d'un bloc

DECLARE (facultatif)

Déclarer les objets PL/SQL à

BEGIN (Obligatoire)

Définir les instructions exécutables (code)

EXCEPTION (facultatif)

Définir les actions en cas d'erreur ou exception

END; (Obligatoire)

Sous Block

Declare
Begin

End;

Structure d'un bloc

1. Section déclaration (facultatif)

- Introduite par le mot clé **DECLARE**
- Contient la description des structures et des variables utilisées dans le bloc

2. Section corps du bloc (Obligatoire)

- Introduite par le mot clé **BEGIN**
- Se termine par le mot clé **END**
- Contient les instructions du programme et éventuellement, à la fin, la section traitement des erreurs

3. Section traitement des erreurs (Facultative)

- Introduite par le mot clé **EXCEPTION**

Syntaxe

[DECLARE]

déclaration des types et variables

BEGIN

corps-du-bloc

clauses SQL

instructions PL/SQL

[EXCEPTION]

traitement-des-erreurs

actions à réaliser quand une exception est levée

END;

/ ← A ajouter dans l'exécution d'un script

Types de blocs

ANONYMOUS

[DECLARE]

BEGIN

instructions
PL/SQL

[EXCEPTION]

END;

PROCEDURE

PROCEDURE name
IS

BEGIN

instructions PL/SQL

[EXCEPTION]

END;

FUNCTION

FUNCTION name
RETURN datatype
IS

BEGIN

instructions PL/SQL

[EXCEPTION]

END;

Exemple

```
DECLARE
```

```
  x VARCHAR2(10) ;
```

```
BEGIN
```

```
  x := 'Bonjour' ;
```

```
  DBMS_OUTPUT.PUT_LINE(x) ;
```

```
END ;
```

```
/
```

Identificateurs

- Identificateur :
 - Variable, curseur, exception, etc.
 - Commence par une lettre
 - Peut contenir : lettres, chiffres, \$, #,
 - Interdits : &, -, /, espace
 - Jusqu'à 30 caractères
 - Insensible à la casse !
- Commentaires :
 - Commentaire sur une seule ligne
 - /* Commentaire sur plusieurs lignes */

- Variables-

Types des variables

1. Les variables **scalaires**: recevant une seule valeur de types prédéfinis correspondants aux types SQL2 ou Oracle: integer, varchar, number,...
2. Les variables de **récupération des colonnes et des lignes** des tables SQL: **%TYPE, %ROWTYPE**
3. Les variables **composites** peuvent contenir plusieurs variables organisées en **collections** ou en **enregistrements**.
 - Constructeur **RECORD**
 - Constructeur **TABLE**

Variables scalaires

Syntaxe

```
identificateur[CONSTANT] typeDeDonnée [NOT  
NULL] [:=|DEFAULT exp];
```

Exemples

DECLARE

```
Dnaissance DATE;  
Ndept NUMBER(2) NOT NULL :=10;  
Ville VARCHAR(13) := 'Tlemcen';  
CodeP CONSTANT NUMBER :=13000;  
VALIDE BOOLEAN NOT NULL DEFAULT FALSE;
```

Variables scalaires

Syntaxe

```
identificateur[CONSTANT] typeDeDonnée [NOT  
NULL] [:=|DEFAULT exp];
```

Règles:

- Adopter les conventions de dénomination d'objets
- Initialiser les constantes et les variables **NOT NULL**
- Initialiser les identifiants en utilisant l'opérateur d'affectation **:=** ou le mot **DEFAULT**
- Déclarer au plus un identifiant par ligne:

~~*i,j:integer;*~~ $\rightarrow$ *i:integer; j:integer;*

Types scalaire

- **CHAR**[(*<taille_max>*)] chaînes de caractères de longueur fixe (max 32767)
- **VARCHAR2**(*<taille_max>*) chaînes de caractères de longueur variable (max 32767)
- **NUMBER** [(*<p>*, *<s>*)] nombres réels, **p** chiffres en tout, **s** après la virgule
- **PLS_INTEGER** prennent moins de place et sont plus rapides que les valeurs de type number et binary_integer
- **DATE**
- **BOOLEAN** trois valeurs possibles: TRUE, FALSE et NULL

Variable %TYPE

%TYPE : permet d'identifier dynamiquement le type d'une variable à partir de :

- la définition d'un attribut de table
- la définition d'une autre variable déclarée précédemment

Syntaxe

```
identificateur nomTable.nomColonne%TYPE;  
identificateur2 identificateur1%TYPE
```

Exemple : Soit la table **CLIENT** (num, nom (VARCHAR(20)), ville)

DECLARE

```
Vnom client.nom%TYPE; --Vnom est de type VARCHAR(20)  
Dnaissance DATE;  
DRecrute Dnaissance%TYPE; --DRecrute est de type DATE
```

RQ : NOT NULL ne s'applique pas aux variable déclarées avec %type

Variable %ROWTYPE

%ROWTYPE: permet de travailler au niveau d'un enregistrement (record).
Il est composé d'un ensemble de colonnes.

Exemple : Soit la table **CLIENT** (num, nom, ville)

```
DECLARE
  Vclient  client%ROWTYPE;
BEGIN
  Vclient.num := 23;
  Vclient.nom := 'SAHOULI';
  Vclient.Ville := 'Tlemcen';
  Insert into CLIENT values Vclient;
END;
```

- Les enregistrements sont semblables, en terme de lignes, à la structure de la table client.
- L'accès au champs par la notation pointée: Vclient.ville

Variable RECORD

- Permet de définir des structures composées de données personnalisées, contrairement à %ROWTYPE qui déclare une structure composée de colonnes de table
- équivalent du **struct** en C

Syntaxe

```
TYPE nomRecord IS RECORD  
(nomChamp typeDonnées [[NOT NULL] {:=|DEFAULT} expression]  
[, nomChamp typeDonnées ...]...) ;
```


Variable RECORD

Exemple

DECLARE

TYPE avionAirbus **IS RECORD**

(nserie CHAR(10),

nomAvion CHAR(20),

usine CHAR(20) := 'Usine_B');

r_A320 avionAirbus; ---la variable de type avionAirbus

BEGIN

r_A320.nserie := 'A1' ;

r_A320.nomAvion := 'r_A320-200' ;

.....

END ;/

Variable RECORD

- Un RECORD ne peut pas être stockés dans une table
- Un champ d'un RECORD peut être lui-même un RECORD, ou soit déclaré avec les directives %TYPE ou %ROWTYPE

Exemple

DECLARE

TYPE avionAirbus **IS RECORD**

(nserie CHAR(10),

nomAvion CHAR(20),

usine CHAR(20) := 'Usine_Balagnac');

TYPE vols_rec **IS RECORD**

(aéranef **avionAirbus** ,

dateVol DATE,

CoPilote **Pilote%ROWTYPE**);

Variables Tableaux (Statique)

IS VARRAY permet de définir et de manipuler un tableau de taille prédéfinie

Syntaxe

```
TYPE <nom_type> IS VARRAY (<size>) OF <type>
```

Exemple

```
DECLARE  
TYPE tab_Clients IS VARRAY(10) OF VARCHAR(20);  
tab1 tab_Clients ;  
BEGIN  
tab1(1) := 'Amine';  
tab1(2) := 'Djamel';  
END ; /
```

Variables Tableaux (dynamique)

ISTABLE OF permet de définir et de manipuler un **tableau dynamique** (défini sans dimension initiale).

Syntaxe

```
TYPE nom_type_tableau IS TABLE OF  
{type_scalaire | variable%TYPE | nom_RECORD  
 | nom_table.colonne%TYPE | nom_table%ROWTYPE}  
[NOT NULL]  
[INDEX BY {BINARY_INTEGER | PLS_INTEGER | VARCHAR2  
(taille) )]
```

Variables de substitution

- Passer comme paramètres à un bloc PL/SQL des variables définies sous SQL*Plus ou l'éditeur SQL de sqldeveloper
- préfixer le nom de la variable du symbole « & »
- La directive **ACCEPT...PROMPT** permet la saisie au clavier de variables dans l'interface SQL*Plus

Exemple

```
ACCEPT v_numP PROMPT 'code produit est: '  
ACCEPT v_libelle PROMPT 'libelle du produit: '  
DECLARE  
V_qte NUMBER(6,2) DEFAULT 32;  
BEGIN  
INSERT INTO produit VALUES ('&v_numP',  
'&v_libelle', v_qte);  
END;
```

Variables de session

- Stockées dans la **session utilisateur** et utilisées pour partager des valeurs et des paramètres entre différents blocs pendant la même session.
- préfixer le nom de la variable du symbole « : »
- La directive **VARIABLE** permet de définir la variable en debut du bloc
- L'affichage de la variable se fait par la directive **PRINT**

Exemple

```
VARIABLE g_compteur NUMBER;  
DECLARE  
v_compteur NUMBER(3) := 99;  
BEGIN  
:g_compteur := v_compteur+ 2;  
END;  
/  
print g_compteur;
```

Opérateurs

Type	Opérateurs
Arithmétique	+, -, *, /, MOD, **
Comparaison	=, !=, <>, >, <, >=, <=, IS NULL, IS NOT NULL
Logique	AND, OR, NOT
Concaténation	

Extraire les données de la base (INTO)

Syntaxe

```
SELECT liste INTO { nomVariablePLSQL [,  
    nomVariablePLSQL ] ... ] nomRECORD } FROM nomTable..;
```

- La clause INTO est obligatoire et permet de préciser les noms des variables (PL/SOL, globales ou hôtes) contenant les valeurs renvoyées par la requête.
- Définir une variable par colonne sélectionnée en respectant l'ordre
- Une requête SELECT .. INTO doit renvoyer un seul enregistrement
- Une requête qui **renvoie plusieurs enregistrements**, ou qui n'en renvoie **aucun**, génère une erreur PL/SQL : **TOO_MANY_ROWS**, **NO_DATA_FOUND**

Extraire les données de la base (INTO)

Exemple

```
VARIABLE g_nom CHAR(20 );  
DECLARE  
rty_pilote Pilote%ROWTYPE;  
v_compa    Pilote.compta%Type;  
BEGIN  
Select * into rty_pilote from Pilote  
  where brevet= 'PL_1'; -- Extraction d'un enregistrement  
SELECT compa into v_compa FROM Pilote  
  WHERE brevet= 'PL-2'; -- Extraction de la valeur d'une  
    colonne  
SELECT nom into :g_nom FROM Pilote  
WHERE brevet= 'PL-2';  -- Extraction de la valeur d'une  
    colonne dans la variable globale  
END;
```

Le package DBMS_OUTPUT

- Ce paquetage assure **la gestion des entrées/sorties** de blocs ou sous-programmes PUSQL
- Il contient plusieurs procédures parmi elles

Procédure	Explication
ENABLE (taille_ tampon IN INTEGER DEFAULT 200 0)	Active le buffer avec une taille spécifiée.
DISABLE	Désactivation du paquetage dans la session.
PUT (ligne);	Affiche la ligne sans saut de ligne
NEW_LINE	Insère un saut de ligne dans la sortie.
PUT_LINE (ligne) ;	Affiche la ligne avec un saut de ligne. Cad un PUT suivi d'un NEW_LINE

Le package DBMS_OUTPUT

Exemple

```
SET SERVEROUTPUT ON
DECLARE v_nbrPil NUMBER;
BEGIN
DBMS_OUTPUT.PUT_LINE('Nous sommes le: ' || SYSDATE);
DBMS_OUTPUT.PUT_LINE('La racine de 2 = ' || SQRT (2));
SELECT count(*) INTO v_nbrPil FROM Pilote;
DBMS_OUTPUT.PUT_LINE('Il y a ' || v_nbrPil || 'pilotes
dans la table');
END;
```

- **RQ** : Au niveau de l'interface SQL*Plus ou SQL Developer, le paquetage doit être activé dans la session avec la commande **SET SERVEROUTPUT ON**
Une fois exécutée, cette option reste valable durant toute la session SQL*Plus.

- Structures de contrôle -

Structures de contrôle

- Structures Conditionnelles
 - If then else
 - Case when
- Structures Répétitives
 - While
 - Loop
 - For

Structure IF THEN

- Trois formes de IF

1. IF..THEN (si-alors)

```
IF condition THEN
    instructions;
END IF;
```

2. IF..THEN..ELSE (avec le sinon)

```
IF condition THEN
    instructions;
ELSE
    instructions;
ENDIF;
```

3. IF..THEN.. ELSIF (imbrications de conditions)

```
IF condition1 THEN instructions;
ELSEIF condition2 THEN instructions;
ELSE
    instructions;
ENDIF;
```

Structure IF THEN

Exemple

```
SET SERVEROUTPUT ON
DECLARE
  v_téléphone CHAR(14) NOT NULL := '05-76-85-14-89';
BEGIN
  IF SUBSTR(v_téléphone,1,2) = '06' THEN
    DBMS_OUTPUT.PUT_LINE ('c''est un portable mobilis !');
  ELSIF SUBSTR(v_téléphone,1,2) = '07' THEN
    DBMS_OUTPUT.PUT_LINE ('c''est un portable DJEZZY!');
  ELSIF SUBSTR(v_téléphone,1,2) = '05' THEN
    DBMS_OUTPUT.PUT_LINE ('c''est un portable Nedjma!');
  ELSE
    DBMS_OUTPUT.PUT_LINE ('c''est un fixe!');
  END IF;
END;
```

Structure CASE

Syntaxe

```
CASE variable  
  WHEN expression1 THEN instructions1;  
  WHEN expression2 THEN instructions2;  
  ...  
  WHEN expression2 THEN instructionsN;  
  [ELSE instructions N+1;]  
END CASE ;
```

- La clause ELSE est optionnelle.
- Si aucune expression n'est valide et le ELSE n'est pas présent, PL/SQL ajoute par défaut l'instruction « ELSE RAISE CASE_NOT_FOUND; » qui lève une exception du même nom

Structure CASE

Exemple

```
SET SERVEROUTPUT ON
DECLARE
    v_téléphone CHAR(14) NOT NULL := '05-76-85-14-89';
    Operateur varchar(8);
BEGIN
    CASE
    WHEN SUBSTR(v_téléphone,1,2)='06' THEN Operateur := 'mobilis';
    WHEN SUBSTR(v_téléphone,1,2)='07' THEN Operateur := 'DJEZZY';
    WHEN SUBSTR(v_téléphone,1,2)='05' THEN Operateur := 'Nedjma';
                                     ELSE Operateur := 'FIXE';
    END CASE;
    DBMS_OUTPUT.PUT_LINE ('c'est un numéro ' || Operateur);
END;
```

Structure répéter (LOOP)

Syntaxe

```
LOOP  
  instructions ;  
EXIT [WHEN condition ;]  
END LOOP ;
```

- La première itération est effectuée quelles que soient les conditions initiales.
- La condition est vérifiée à chaque itération ; si elle est vraie, on quitte la boucle sinon la séquence d'instructions est de nouveau exécutée
- Si aucune condition n'est spécifiée (WHEN condition absent), les instructions sont exécutées **une seule** fois

Structure répéter (LOOP)

Exemple

```
DECLARE
V_numPilote NUMBER(3) := 1;
BEGIN
LOOP
    INSERT into pilote(brevet,COMPA)
    VALUES ('PL_'||to_char(V_numPilote), 'AF') ;
V_numPilote:= V_numPilote+1 ;
EXIT WHEN V_numPilote>= 10 ;
END LOOP ;
END ;
```

Structure tant que (WHILE)

Syntaxe

```
WHILE   Condition LOOP  
    Instructions;  
END LOOP;
```

- La condition est vérifiée au **début** de chaque itération
- tant que la condition est vérifiée, les instructions sont exécutées

Structure tant que (WHILE)

Exemple

```
DECLARE
V_numPilote NUMBER(3) := 1;
BEGIN
WHILE V_numPilote <= 10 LOOP
    INSERT into pilote(brevet,COMPA)
    VALUES ('PL_'||to_char(V_numPilote), 'AF') ;
V_numPilote:= V_numPilote+1 ;
END LOOP;
END ;
```

Structure pour (FOR)

Syntaxe

```
FOR compteur IN [REVERSE]  
    valeurInf .. valeurSup LOOP  
    instructions;  
END LOOP;
```

- **compteur** est une variable locale à la boucle non déclarée
- **valeurInf** et **valeurSup** sont des variables locales déclarées et initialisées ou alors des constantes
- Le pas est fixe à 1, éventuellement en décrétement si **REVERSE**
- Il n'est pas possible de modifier le pas de la variable d'itération dans le corps d'une boucle.

Structure pour (FOR)

Exemple

```
BEGIN
FOR  V_numPilote IN 1..10 LOOP
    INSERT into pilote(brevet,COMPA)
    VALUES ('PL_'||to_char(V_numPilote), 'AF') ;
END LOOP;
END ;
```

Exemple de procédure

Exemple

DECLARE

```
Procedure inserer_pilote (VBrevet in  
pilote.brevet%TYPE, VCOMPA IN pilote.compa%TYPE) as  
BEGIN  
    INSERT into pilote (brevet, COMPA)  
    VALUES (VBrevet, VCOMPA) ;  
END inserer_pilote;
```

BEGIN

```
FOR V_numPilote IN 14..16 LOOP  
    inserer_pilote ('PL_' || to_char(V_numPilote), 'AF');  
END LOOP;  
END ;
```